

AI Loadboard — технічна документація

Архітектура системи, доменна модель, AI-підсистема та інженерні рішення — від того, що вже працює, до того, що спроектовано на масштаб.

Версія документа **0.1** Дата **15.08.2026** Статус продукту **MVP / демо**

Пов'язаний документ **Project Overview (для інвесторів)**

01 Про документ

Це технічна специфікація проекту AI Loadboard — вантажної біржі з прямим з'єднанням вантажовідправника і перевізника, без брокера-посередника. На відміну від інвесторського огляду (`project-documentation.html`), тут немає маркетингового спрощення: документ описує систему так, як її бачить інженерна команда — включно з тим, що ще не побудовано.

Аудиторія: технічні співзасновники, майбутні інженери, що приєднуються до команди, і технічний due diligence з боку інвестора.

Кожен розділ і кожен рядок таблиці позначені міткою зрілості:

MVP · ГОТОВО — реалізовано і працює в поточному демо-каркасі · **ФАЗА 2** — у найближчому спринті розробки · **ФАЗА 3** — заплановано, дизайн ще не заморожений · **R&D** — потребує дослідження/прототипування моделі перед комітом у продукт · **РИЗИК** — відомий технічний борг або відкрите питання.

Документ навмисно не приховує незрілість поточної кодової бази: MVP — це навмисно спрощений транзакційний каркас (реєстрація → публікація вантажу → взяття вантажу → завершення), без жодної AI-функції, зібраний, щоб перевірити базовий цикл shipper↔carrier до інвестиції в ML-шар.

02 Огляд системи та рівень зрілості

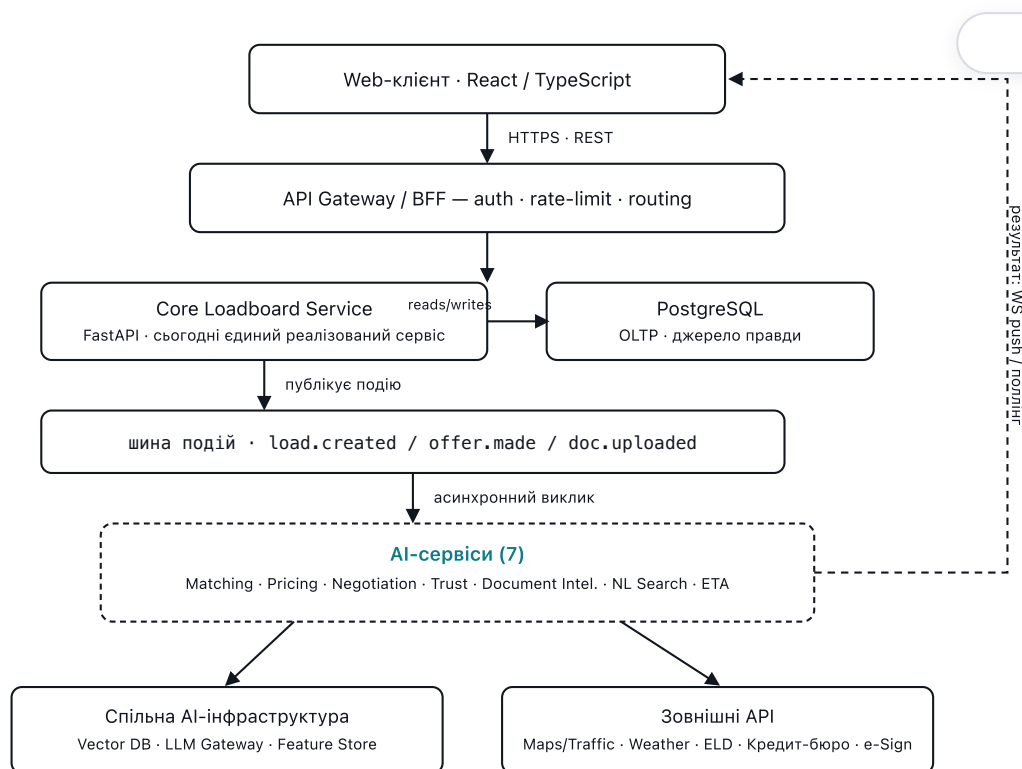
AI Loadboard — двостороння платформа: вантажовідправник публікує вантаж, перевізник бачить його і бере напряду, без брокера. Довгострокове бачення — не лише дошка оголошень, а система, де AI виконує роботу диспетчера й брокера: підбирає вантаж, прогнозує ставку, веде перші переговори, перевіряє контрагента і обробляє документи.

КОМПОНЕНТ	ПОТОЧНИЙ СТАН (MVP)	ЦІЛЬОВИЙ СТАН
Клієнт	React SPA, один екран, ручний <code>fetch</code>	React SPA + мобільний застосунок (React Native), кешування запитів, realtime через WebSocket
Бекенд	Один монолітний FastAPI-сервіс	Core-сервіс + вісім AI-сервісів, розв'язаних через шину подій
База даних	Один PostgreSQL, 2 таблиці (<code>users</code> , <code>loads</code>)	PostgreSQL (OLTP) + pgvector/векторна БД + Redis (кеш/черги) + об'єктне сховище документів
Автентифікація	Email + пароль, JWT без refresh	JWT + refresh-токени, MFA, SSO для enterprise-акаунтів
Матчинг вантажів	Лінійний список, сортування за датою	ML-ранжування під профіль перевізника
AI-функції	Відсутні	7 AI-сервісів (розділ 7)
Інфраструктура	Docker Compose, один хост	Kubernetes, кілька середовищ, автоскейлінг
Спостережуваність	Немає — лише логи <code>uvicorn</code> у stdout	Метрики, трейсинг, структуровані логи, алертинг, AI-eval пайплайни

Навмисне рішення, не недогляд. AI не додається «зверху» на готову дошку оголошень — уся архітектура нижче спроектована так, щоб AI-сервіси підключалися як окремі, незалежно деплойовані споживачі подій, а не як виклики всередині монолітного контролера. Це головна архітектурна ставка проєкту.

03 Архітектура верхнього рівня

Система розділена на дві половини з різними вимогами до надійності й латентності. **Core Loadboard Service** — транзакційне ядро (реєстрація, вантажі, статуси, платежі в майбутньому): має бути швидким, консистентним, з жорсткими SLA. **AI-сервіси** — обчислювально важкі, залежать від зовнішніх LLM/ML-провайдерів, можуть відповідати з затримкою в секунди і не повинні блокувати транзакційний шлях. Тому вони зв'язані асинхронно через шину подій, а не прямими синхронними викликами.



Легенда: суцільна лінія — синхронний запит; пунктир — асинхронний результат, що повертається клієнту.

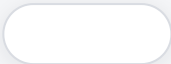
Мал. 1 — Транзакційне ядро і AI-сервіси розв'язані шиною подій: Core пише у Postgres і публікує подію, AI-сервіси споживають її асинхронно й повертають результат клієнту окремим каналом (WS/поллінг), не блокуючи основний запит.

Наразі реалізований лише прямокутник **Core Loadboard Service ↔ PostgreSQL** у верхній половині схеми — без API Gateway, без шини подій, без жодного AI-сервісу. Решта — цільова архітектура, під яку вже зараз проєктується доменна модель (розділ 5) і контракти подій.

Чому шина подій, а не прямі виклики

- **Ізоляція збоїв.** Якщо LLM-провайдер деградує чи впаде, публікація вантажу і взяття вантажу продовжують працювати — це різні відмовостійкі домени.
- **Незалежне масштабування й деплой.** Negotiation Agent можна викотити окремо від Core, дати йому свій пул воркерів і GPU/API-бюджет, не чіпаючи транзакційний код.
- **Заміна моделі без даунтайму ядра.** Провайдер LLM або алгоритм скорингу можна змінити, підключивши нову версію споживача до тієї самої події.
- **Аудит за замовчуванням.** Кожна подія — це запис у журналі: хто що запропонував і коли, важливо для спорів «AI щось пообіцяв контрагенту».

04 Технологічний стек



Клієнт

ТЕХНОЛОГІЯ	РОЛЬ	СТАТУС
React 18 + TypeScript	SPA, компонентний UI	MVP
Vite	Dev-сервер і білд	MVP
Ручний <code>fetch</code> + React Context	Доступ до API та стан автентифікації (<code>AuthContext.tsx</code>)	MVP
TanStack Query	Кешування запитів, повторні спроби, інвалідація	ФАЗА 2
WebSocket-клієнт	Realtime-оновлення статусів вантажу, офери AI-агента	ФАЗА 2
React Native	Мобільний застосунок для перевізників (сповіщення, GPS)	ФАЗА 3

Бекенд і дані

ТЕХНОЛОГІЯ	РОЛЬ	СТАТУС
Python 3.12 + FastAPI	REST API, валідація через Pydantic	MVP
SQLAlchemy 2.x	ORM, декларативні моделі	MVP
PostgreSQL 16	Основне сховище (OLTP)	MVP
python-jose + passlib/bcrypt	JWT, хешування паролів	MVP
Redis	Кеш сесій, rate-limit лічильники, черга задач (Celery/RQ broker)	ФАЗА 2
pgvector (розширення Postgres)	Векторний пошук для NL-запитів та матчингу — старт без окремої векторної БД	ФАЗА 2
Kafka / Redis Streams	Шина подій між Core і AI-сервісами	ФАЗА 3
S3-сумісне об'єктне сховище	Зберігання документів (BOL/POD/rate confirmation)	ФАЗА 2
Alembic	Міграції схеми БД (замість <code>create_all</code> на старті)	ФАЗА 2

AI / ML

ТЕХНОЛОГІЯ	РОЛЬ	СТАТУС
LLM Gateway (LiteLLM-подібний шар)	Єдина точка виклику моделей різних провайдерів, з фолбеком і лічильником вартості	R&D
Claude / GPT-клас моделей (function calling)	Negotiation Agent, NL Search, структурування документів	R&D
Sentence-embedding модель (напр. клас <code>bge</code> / <code>e5</code>)	Векторизація вантажів і запитів для матчингу й пошуку	R&D
Gradient boosting (LightGBM/XGBoost)	Прогноз ринкової ставки, ETA, trust-скоринг — табличні дані, пояснюваність важливіша за глибокі мережі	R&D
OCR + layout-модель (напр. клас LayoutLM/Texttract)	Розпізнавання BOL/POD/rate confirmation	R&D
MLflow / реєстр моделей	Версіонування моделей, відтворюваність	ФАЗА 3

Інфраструктура та спостережуваність

ТЕХНОЛОГІЯ	РОЛЬ	СТАТУС
Docker + Docker Compose	Локальний запуск трьох контейнерів (db/backend/frontend)	MVP
Kubernetes (керований — EKS/GKE)	Оркестрація в проді, автоскейлінг AI-воркерів окремо від Core	ФАЗА 3
GitHub Actions	CI: лінт, тести, білд образів, деплой	ФАЗА 2
Terraform	Інфраструктура як код (мережа, БД, кластер)	ФАЗА 3
OpenTelemetry + Prometheus/Grafana	Метрики й трейсинг наскрізних запитів, включно з AI-викликами	ФАЗА 3
Sentry	Трекінг помилок фронтенду і бекенду	ФАЗА 2

05 Доменна модель даних

Поточна схема — дві таблиці, свідомо мінімальні для MVP:

```
-- backend/app/models.py — те, що реально існує сьогодні

users(id, email UNIQUE, hashed_password, company_name, role ENUM(shipper|carrier), created_at)

loads(id, title, origin, destination, equipment_type DEFAULT 'Dry Van',
      weight_lbs, price_usd NUMERIC(10,2), shipper_name, carrier_name NULLABLE,
      status ENUM(open|accepted|completed), created_at)
```

Помітний розрив зараз: `loads.shipper_name` і `carrier_name` — вільні текстові поля, не зовнішні ключі на `users`. Вантаж технічно не прив'язаний до автентифікованого користувача, який його опублікував чи взяв. Це працює для демо, але перше, що змінюється в Фазі 2 — `shipper_id` / `carrier_id` як FK з перевіркою власності на рівні API. Деталі — розділ 17.

Цільова схема (Фаза 2–3)

Розширення додає компанії (для trust-скорингу й MC/DOT-номерів), офери та історію переговорів, документи з результатами розпізнавання, знімки прогнозів ставки:

```

CREATE TABLE companies (
  id          BIGSERIAL PRIMARY KEY,
  legal_name  TEXT NOT NULL,
  mc_number   TEXT UNIQUE,      -- Motor Carrier number
  dot_number  TEXT UNIQUE,
  trust_score NUMERIC(4,1),     -- кеш останнього скору, 0-100
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE offers (
  id          BIGSERIAL PRIMARY KEY,
  load_id     BIGINT REFERENCES loads(id),
  carrier_id  BIGINT REFERENCES users(id),
  price_usd   NUMERIC(10,2) NOT NULL,
  status      TEXT NOT NULL DEFAULT 'pending', -- pending|countered|accepted|rejected|e
  source      TEXT NOT NULL DEFAULT 'human',   -- human|ai_agent
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE negotiation_messages (
  id          BIGSERIAL PRIMARY KEY,
  offer_id    BIGINT REFERENCES offers(id),
  sender      TEXT NOT NULL,   -- shipper|carrier|ai_agent
  channel     TEXT NOT NULL,   -- chat|email|sms|voice_transcript
  body        TEXT NOT NULL,
  proposed_price NUMERIC(10,2),
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE trust_scores (
  company_id  BIGINT REFERENCES companies(id),
  score       NUMERIC(4,1) NOT NULL,   -- 0-100
  factors     JSONB NOT NULL,          -- {"payment_history":..., "insurance_valid":...
  model_version TEXT NOT NULL,
  computed_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (company_id, computed_at)
);

CREATE TABLE market_rate_snapshots (
  lane        TEXT NOT NULL,   -- нап. 'DAL-HOU'
  equipment_type TEXT NOT NULL,
  predicted_at TIMESTAMPTZ NOT NULL,
  horizon_days INT NOT NULL,
  predicted_usd NUMERIC(10,2) NOT NULL,
  confidence_low NUMERIC(10,2),
  confidence_high NUMERIC(10,2),
  model_version TEXT NOT NULL
);

```



```
CREATE TABLE documents (  
  id          BIGSERIAL PRIMARY KEY,  
  load_id     BIGINT REFERENCES loads(id),  
  doc_type    TEXT NOT NULL,    -- bol|pod|rate_confirmation|insurance_coi  
  storage_uri  TEXT NOT NULL,    -- s3://...  
  extraction  JSONB,            -- структуровані поля з IDP-пайплайну  
  confidence   NUMERIC(4,3),  
  review_status TEXT NOT NULL DEFAULT 'auto', -- auto|needs_review|reviewed  
  created_at   TIMESTAMPTZ DEFAULT now()  
);
```

`loads.embedding vector(768)` додається як колонка pgvector для семантичного пошуку й матчингу (розділ 7.3/7.6) — окрема векторна БД не потрібна, поки обсяг вантажів не вимагає ANN-індексів поза Postgres.

06 Сервіси бекенду

СЕРВІС	ВІДПОВІДАЛЬНІСТЬ	КЛЮЧОВА ТЕХНІКА	СТАТУС
Core Loadboard Service	Користувачі, вантажі, статуси, офери, оплата (майбутнє)	FastAPI + Postgres	MVP
Matching Service	Ранжування відкритих вантажів під профіль і історію перевізника	pgvector + gradient-boosted ranking	R&D
Pricing Service	Прогноз ринкової ставки на маршруті на N днів вперед	LightGBM / часові ряди, знімки в <code>market_rate_snapshots</code>	R&D
Negotiation Service	LLM-агент, що веде першу комунікацію й торгується в заданих межах	LLM function calling + guardrails (розділ 7.1)	R&D
Trust Service	Скоринг надійності контрагента, флаги подвійного брокериджу	Табличний класифікатор + зовнішні перевірки (FMCSA, страхові)	R&D
Document Intelligence Service	OCR і структурування BOL/POD/rate confirmation	OCR + layout-модель + LLM-екстракція (розділ 7.2)	R&D
Search Service	Пошук вантажів природною мовою	LLM parsing → структурований фільтр + векторний пошук	R&D
ETA Service	Оцінка часу прибуття з урахуванням трафіку/погоди/HOS	Регресія + телематика/трафік API	R&D
Notification Service	Push/email/SMS про зміну статусу вантажу, нові офери	WebSocket + черга задач	ФАЗА 3

Усі AI-сервіси — окремі деплойменти, що читають зі спільної шини подій і пишуть результат назад у Postgres через власний вузький API-контракт (не мають прямого доступу до таблиць одне одного). Це свідомо ближче до модульного моноліту з чіткими межами, ніж до «мікросервісів заради мікросервісів» — команда з 2–5 інженерів не обслужить 9 незалежних репозиторіїв; межі проведені так, щоб їх можна було фізично розрізати пізніше без переписування доменної логіки.

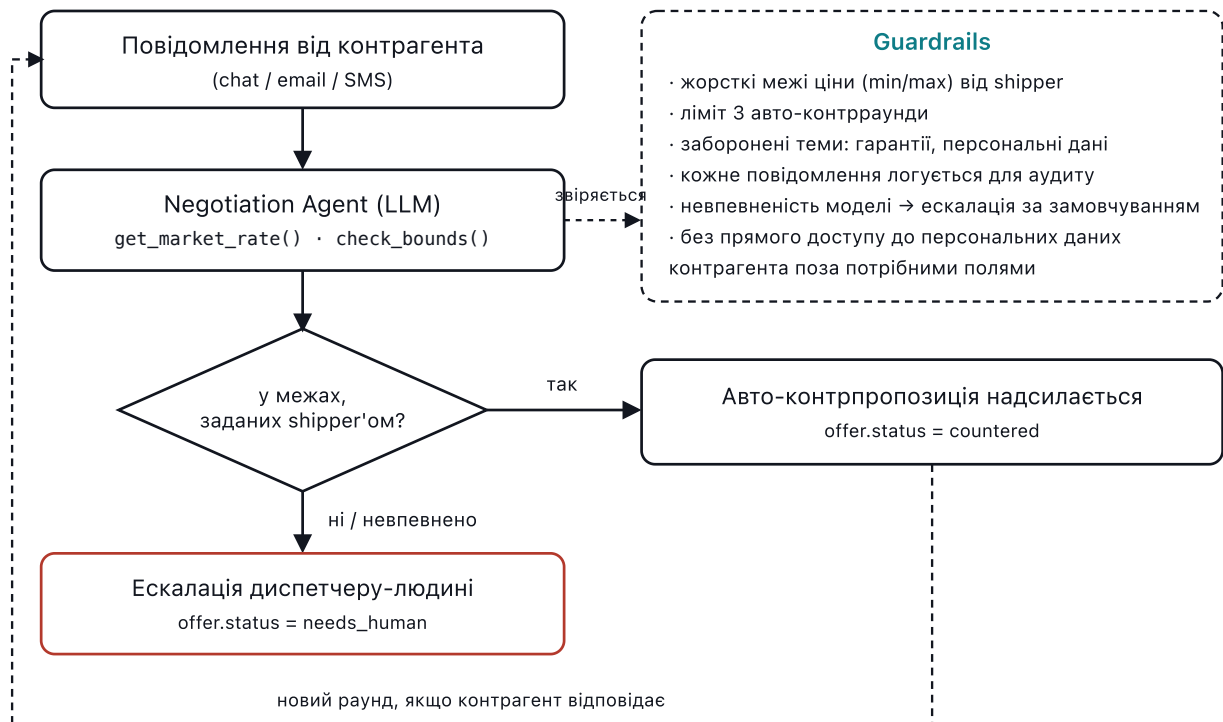
07 AI/ML підсистема

Сім продуктових AI-функцій зі спільною інфраструктурою. Спільний принцип для всіх: **модель ніколи не є єдиною точкою відмови** — кожен сервіс має детерміністичний fallback (порожній список рекомендацій, останню відому ставку, ескалацію людині), і жоден AI-виклик не блокує основний транзакційний шлях (розділ 3).

ФУНКЦІЯ	ПІДХІД	КЛЮЧОВІ ДАНІ	ЛЮДИНА В ЦИКЛІ
Розумний підбір вантажу	Ранжування (embeddings + gradient boosting)	Маршрут, порожній пробіг, історія прийнятих вантажів	Ні — лише сортування, вибір за перевізником
Прогноз ринкової ставки	Часові ряди / gradient boosting	Історичні ставки по лейнах, паливо, сезонність, попит/пропозиція	Ні — довірчий інтервал показується явно
AI-асистент переговорів	LLM-агент із function calling	Межі ціни від shipper, історія лейну, повідомлення контрагента	Так — ескалація за межами довіри (розділ 7.1)
Скоринг надійності	Табличний класифікатор + правила	Історія платежів, FMCSA-дані, страхові документи, флаги шахрайства	Так — низький скор блокує авто-матчинг, потребує ручного review
Автоматизація документів	OCR + LLM-екстракція	Скан/фото BOL, POD, rate confirmation	Так — нижче порогу впевненості (розділ 7.2)
Пошук природною мовою	LLM parsing → фільтр + векторний пошук	Вільний текстовий запит користувача	Ні — детерміністичний фолбек на форму-фільтр
Прогноз ETA	Регресія на телематиці/трафіку	GPS перевізника, трафік, погода, HOS-ліміти водія	Ні

7.1 AI-асистент переговорів

Найризикованіша функція продукту: модель веде реальну комунікацію від імені платформи і може «пообіцяти» ціну. Тому це єдиний AI-сервіс із жорстким контуром безпеки, а не просто LLM-



Мал. 2 — Агент пропонує ціну лише в межах, які задав shipper заздалегідь; вихід за межі або невпевненість моделі завжди веде до ескалації людині, ніколи не до «згоди» AI.

Чому саме так: LLM за конструкцією не гарантує дотримання числових меж лише інструкцією в промпті — тому перевірка меж винесена в детерміністичний код (`check_bounds()`), а не довірена моделі. Агент може пропонувати формулювання і тон, але не може обійти хардкоджену перевірку суми.

7.2 Автоматизація документообігу

Пайплайн розпізнавання BOL/POD/rate confirmation: `upload` → `OCR/layout` → `LLM-структурування у JSON` → `перевірка впевненості` → `авто-збереження або черга ручної перевірки`. Поле `confidence` з таблиці `documents` (розділ 5) — це мінімум `confidence` по обов'язкових полях (сума, дата, підпис/номер вантажу); нижче порогу (напр. 0.90) документ іде в чергу ручної перевірки замість автоматичного застосування. Кожне ручне виправлення записується як приклад для донавчання/промпт-тюнінгу — цикл зворотного зв'язку описаний у розділі 13.

7.3–7.7 Інші AI-сервіси — принципи проектування

Matching (7.3)

Ембединги вантажу (маршрут, тип кузова, вага, час) + профіль перевізника (домашня база, вподобані лейни, історія прийняття/відмов) → cosine-подібність як базовий сигнал, gradient boosting поверх — для калібрування під фактичну конверсію в прийняття.

Pricing (7.4)

Прогноз на лейн-рівні (origin-dest-equipment), горизонт 1–7 днів, з довірчим інтервалом, а не точковим числом — платформа явно показує невпевненість, а не видає її за факт.

Trust Scoring (7.5)

0–100 з поясненими факторами (JSONB `factors`), не «чорна скринька» — перевізник/shipper бачить, чому скор такий, і може оскаржити. Низький скор не банить автоматично, а знижує пріоритет у матчингу і додає попередження.

NL Search (7.6)

LLM переводить вільний запит («рефрижератор із Далласа цього тижня до 900») у структурований фільтр + опційний векторний пошук за смисловою близькістю; завжди є детерміністичний фолбек на форму з фільтрами, якщо парсинг непевний.

ETA (7.7)

Регресія поверх GPS-телематики перевізника (де є доступ), трафіку, погоди і залишку годин водіння (HOS) — точність деградує пояснено (ширший інтервал), якщо телематика недоступна, а не мовчки.

Спільне для всіх

Кожна модель має версію (`model_version`), логується вхід/вихід для офлайн-оцінки, і має задокументований fallback на детермінізм при відмові.

7.8 Спільна AI-інфраструктура

- **LLM Gateway** — єдина точка виклику моделей: абстрагує провайдера (можливість перемкнутись між моделями без зміни коду сервісів), рахує вартість tokenів на запит, застосовує тайм-аут і fallback на меншу/дешевшу модель при відмові основного провайдера.
- **Редагування PII перед відправкою назовні.** Персональні дані водія/контактні дані контрагента маскуються перед тим, як текст іде до стороннього LLM-провайдера, і відновлюються після відповіді — вимога для документообігу з ПІБ водіїв і номерами телефонів.

- **Реєстр промптів і eval-набори.** Кожен промпт версіонується; зміна промпту прогонюється проти зафіксованого набору реальних кейсів (regression suite для AI, а не лише для викаткою).
- **Canary rollout моделей.** Нова версія моделі/промпту спершу отримує невелику частку трафіку, порівнюється за метриками якості й вартості, і лише потім масштабується на 100%.
- **Бюджет вартості й circuit breaker.** Ліміт витрат на AI-виклики за період; при перевищенні — деградація до дешевших/детерміністичних шляхів, а не мовчазне зростання рахунку.
- **Feature Store.** Спільні ознаки (історія лейну, поведінка перевізника) обчислюються один раз і перевикористовуються Matching, Pricing і Trust сервісами замість дублювання логіки.

08 API

Конвенції

- Базовий шлях `/api`, JSON тіла запитів/відповідей, помилки — `{"detail": "..."} (стандартний формат FastAPI/Pydantic).`
- Автентифікація — заголовок `Authorization: Bearer <JWT>`.
- Публічна специфікація автогенерується FastAPI: `/docs` (Swagger UI) та `/openapi.json`.
- Заплановано на Фазу 2: версіонування шляху (`/api/v1/...`), курсорна пагінація (`?cursor=`) для списків, ідемпотентні ключі для `POST /offers`.

Реалізовані ендпоінти (MVP)

МЕТОД	ШЛЯХ	ОПИС	AUTH
GET	/api/health	Перевірка живості сервісу	—
POST	/api/auth/register	Реєстрація (email, пароль, компанія, роль)	—
POST	/api/auth/login	Вхід, видає JWT на 7 днів	—
GET	/api/auth/me	Поточний користувач за токеном	Bearer
GET	/api/loads	Список вантажів, опційний фільтр ?status=	—
POST	/api/loads	Публікація вантажу	— МАЄ БУТИ BEARER
POST	/api/loads/{id}/accept	Взяти вантаж напряму	— МАЄ БУТИ BEARER
POST	/api/loads/{id}/complete	Позначити завершеним	— МАЄ БУТИ BEARER

Розбіжність із README: публічний опис демо каже «доступно тільки авторизованим shipper/carrier», але в коді (`main.py`) ці три роути не мають `Depends(get_current_user)` і не перевіряють роль — сьогодні їх може викликати будь-хто без токена. Це перший пункт у розділі 17.

Заплановані ендпоінти (Фаза 2–3)

МЕТОД	ШЛЯХ	ОПИС
GET	/api/loads/{id}/matches	AI-рекомендації вантажів під профіль перевізника
POST	/api/search	Пошук вантажів природною мовою
GET	/api/rates/forecast	Прогноз ставки за лейном/типом кузова/датою
POST	/api/loads/{id}/offers	Створити офер (людиною або AI-агентом)
POST	/api/offers/{id}/counter	Контрпропозиція
GET	/api/companies/{id}/trust-score	Скор надійності з поясненими факторами
POST	/api/loads/{id}/documents	Завантажити BOL/POD/rate confirmation
GET	/api/loads/{id}/eta	Прогноз часу прибуття
POST	/api/auth/refresh	Оновити access-токен за refresh-токеном
WS	/ws/notifications	Realtime-сповіщення: нові офери, зміни статусу

09 Автентифікація, авторизація й ролі

Сьогодні: email+пароль → `bcrypt` -хеш → JWT (HS256, `sub` =email, термін дії 7 днів) з `python-jose`, без refresh-токена й без відкликання (вихід — лише видалення токена з `localStorage` на клієнті, сам токен лишається дійсним до сплину терміну).

РОЛЬ	МОЖЕ СЬОГОДНІ (ЗА НАМІРОМ)	ЦІЛЬ (ФАЗА 2+)
shipper	Публікувати вантаж	+ бачити оферти, вести переговори, бачити trust-скор перевізника
carrier	Брати відкритий вантаж	+ AI-рекомендації, робити контрпропозиції, завантажувати POD
dispatcher (enterprise)	—	Керує кількома вантажівками/водіями від імені carrier-компанії
admin	—	Модерація, перегляд журналу аудиту, ручний review документів/скорингу

Цільові покращення: `access_token` (15 хв) + `refresh_token` (httpOnly cookie, з ротацією).
відкликання через deny-list у Redis, MFA для enterprise-акаунтів, SSO (OAuth2/OIDC) д. `shipper`-клієнтів, окремі API-ключі з обмеженими scope для інтеграцій (TMS-конектори).

10 Безпека

ОБЛАСТЬ	ПОТОЧНИЙ СТАН	ЦІЛЬ
Секрети	<code>JWT_SECRET</code> зі змінної середовища, дефолт <code>"dev-secret-change-me"</code> у коді	Секрет-менеджер (Vault/AWS Secrets Manager), без дефолтів у коді
CORS	<code>allow_origins=["*"]</code> , всі методи/заголовки	Явний allowlist доменів продакшн-фронтенду
Rate limiting	Відсутній	На рівні API Gateway, окремо жорсткіше на <code>/auth/*</code>
Авторизація на рівні ресурсу	Відсутня (розділ 17)	Перевірка власності (shipper бачить/редагує лише свої вантажі) на кожному mutating-ендпоінті
PII та документи	Н/З — документів ще немає	Шифрування в спокої (S3 SSE), маскування PII перед LLM-викликами (розділ 7.8)
Аудит	Відсутній	<code>audit_log</code> : хто/що/коли, окремо — усі AI-повідомлення від імені платформи (розділ 7.1)
Залежності	Не скануються	Dependabot/ <code>pip-audit</code> у CI, блокування критичних CVE
Пентест	Не проводився (демо-стадія)	Перед публічним запуском і раз на рік/після значних змін

11 Інфраструктура та DevOps

Сьогодні: `docker-compose.yml` піднімає три контейнери — `db` (Postgres 16 з healthcheck), `backend` (FastAPI, ретраї підключення до БД при старті), `frontend` (Vite dev-сервер). Один хост, без окремих середовищ.

СЕРЕДОВИЩЕ	ПРИЗНАЧЕННЯ	СТАТУС
local	Розробка через <code>docker compose up</code>	MVP
staging	Перевірка перед релізом, синтетичні дані	ФАЗА 2
production	Kubernetes, окремі node pool для Core і AI-воркерів (AI-воркери — з доступом до GPU/вищим CPU-лімітом і власним автоскейлінгом за чергою подій)	ФАЗА 3

CI/CD (Фаза 2): GitHub Actions — лінт (ruff/eslint) → тести → білд Docker-образів → пуш у реєстр → деплой у staging автоматично, у production через ручне затвердження. **Резервне копіювання:** щоденний снапшот Postgres + point-in-time recovery до 7 днів; документи в об'єктному сховищі — versioning увімкнено для захисту від випадкового перезапису.

12 Продуктивність і масштабованість

- Кешування.** Redis для «гарячого» списку відкритих вантажів і результатів прогнозу ставки (TTL кілька хвилин — прогноз не повинен перераховуватись на кожен рендер списку).
- Асинхронні задачі.** Важкі AI-виклики (переговори, OCR) йдуть через чергу (Celery/RQ поверх Redis, пізніше — Kafka), не в request-response циклі API.
- Індекси БД.** `loads(status, created_at)` складений індекс для основного списку; `loads(origin, destination, equipment_type)` для матчингу; pgvector HNSW-індекс на `embedding`.
- Read-репліки.** Читання списку вантажів і аналітики — з репліки, запис — лише на primary, коли навантаження це виправдає.
- CDN.** Статика фронтенду й завантажені документи (через підписані URL) — за CDN, не проксуються бекендом.

МЕТРИКА	ЦІЛЬОВИЙ БЮДЖЕТ
Список вантажів, p95	< 200 мс
Публікація вантажу, p95	< 300 мс
AI-матчинг (асинхронний), p95	< 3 с
Перший хід Negotiation Agent, p95	< 8 с
Доступність Core API	99.9%

13 Спостережуваність

Сьогодні — лише stdout-логи `uvicorn`, нічого структурованого. Цільовий стек:

- **Логи** — структуровані (JSON), з `request_id`, що проходить через Gateway → Core → подію → AI-сервіс, для наскрізного трасування одного запиту користувача.
- **Метрики** — Prometheus: латентність по ендпоінтах, довжина черги подій, вартість AI-викликів у доларах на добу.
- **Трейсинг** — OpenTelemetry, окремо позначені AI-спани (модель, кількість токенів, латентність провайдера).
- **AI-специфічна спостережуваність:** eval-пайплайн на sample реальних кейсів після кожної зміни промпту/моделі; дрейф якості (частка ескалацій Negotiation Agent людині — раптове зростання сигналізує про регресію); черга людського review для документів і скорингу як явний UI, а не побічний ефект.
- **Цикл зворотного зв'язку.** Кожне ручне виправлення (диспетчер поправив офер AI, людина виправила поле документа) записується як приклад для наступного eval-набору й потенційного донавчання/промпт-тюнінгу.

14 Зовнішні інтеграції

КАТЕГОРІЯ	НАВІЩО	ВИКОРИСТОВУЄТЬСЯ В
Картографія / трафік	Відстань, час у дорозі, затори	ETA, Matching (порожній пробіг)
Погода	Коригування ETA, ризик затримки	ETA
ELD / телематика перевізника	Реальна GPS-позиція, HOS-ліміти водія	ETA, Trust
FMCSA / кредитні бюро	MC/DOT-верифікація, страхове покриття, історія претензій	Trust Scoring, онбординг компанії
e-Signature	Підписання rate confirmation без обміну PDF поштою	Document Intelligence, офери
Платежі / факторинг	Швидка оплата перевізнику після POD (майбутня монетизація)	Завершення вантажу
SMS / email провайдер	Сповіщення, канал для Negotiation Agent	Notification, Negotiation
DAT/Truckstop (публічні API, де доступно)	Холодний старт ринкових даних для Pricing до накопичення власної історії	Pricing Service

15 Тестування та якість

РІВЕНЬ	ІНСТРУМЕНТ	ЩО ПОКРИВАЄ	СТАТУС
Unit (бекенд)	pytest	Схеми, бізнес-правила (напр. переходи статусу вантажу)	ФАЗА 2
Unit (фронтенд)	Vitest + Testing Library	Компоненти, AuthContext	ФАЗА 2
Інтеграційні	pytest + тестовий Postgres у Docker	Ендпоінти API наскрізь через БД	ФАЗА 2
E2E	Playwright	Реєстрація → публікація → взяття вантажу	ФАЗА 3
AI-eval	Власний харнес поверх зафіксованого набору кейсів	Якість Negotiation/Matching/Trust — регресія перед кожним релізом промпту чи моделі	R&D
Навантажувальні	k6/Locust	Латентність під пік-навантаженням, черга подій під бекпресурою	ФАЗА 3

Сьогодні тестів немає — жодного файлу під `backend/tests` чи `frontend/src/**/*.test.*`. Перший пріоритет Фази 2: покрити перехід статусів вантажу (`open→accepted→completed`) і авторизацію ендпоінтів, оскільки саме там зараз відомий розрив (розділ 17).

16 Технічна дорожня карта

ФАЗА	ТЕХНІЧНА ЦІЛЬ	АІ-ФІЧІ, ЩО ВМИКАЮТЬСЯ
MVP ГОТОВО	Транзакційний каркас: auth, вантажі, статуси	—
Фаза 2 3–4 МІС	Авторизація на рівні ресурсу, тести, CI, черга задач, об'єктне сховище, pgvector	Розумний підбір вантажу, пошук природною мовою (базова версія)
Фаза 3 5–9 МІС	Kubernetes, шина подій, спостережуваність, мобільний застосунок	Прогноз ставки, скоринг надійності, автоматизація документів, ETA
Фаза 4 R&D	Multi-region, SSO для enterprise, платіжний рейл	AI-асистент переговорів (найризикованіша фіча — останньою, після довіри користувачів до простіших AI-функцій)

Порядок навмисний: Negotiation Agent — найцінніша, але й найризикованіша фіча (модель говорить від імені платформи з реальними грошима на кону). Вона йде останньою, коли інфраструктура guardrails, аудиту й ескалації (розділ 7.1, 13) вже перевірена на менш ризикованих AI-функціях.

17 Технічний борг і відомі обмеження

#	ПРОБЛЕМА	ВПЛИВ	ПРІОРИТЕТ
1	<code>POST /api/loads</code> , <code>/accept</code> , <code>/complete</code> не перевіряють автентифікацію чи роль, попри те що README це обіцяє	Будь-хто без токена може публікувати/забирати/закривати вантажі	P0
2	<code>loads.shipper_name</code> / <code>carrier_name</code> — вільний текст, не FK на <code>users</code>	Неможливо перевірити власність вантажу; дані легко підробити	P0
3	JWT без refresh і без відкликання, дефолтний секрет у коді	Вкрадений токен дійсний до 7 днів; ризик при випадковому деплої з дефолтним <code>JWT_SECRET</code>	P1
4	CORS <code>allow_origins=["*"]</code>	Прийнятно для демо, неприйнятно для продакшну з реальними даними	P1
5	Схема БД створюється через <code>Base.metadata.create_all</code> , без міграцій	Немає керованого шляху еволюції схеми в продакшн-даних	P1
6	Немає тестів (бекенд і фронтенд)	Регресії не ловляться до продакшну	P1
7	Немає rate limiting на <code>/api/auth/*</code>	Вразливість до brute-force підбору пароля	P2
8	AI-сервіси — поки що специфікація, не код	Основна цінність продукту для інвестора ще не побудована	ВІДКРИТО

Пункти 1–2 — перше, що варто закрити перед будь-яким публічним доступом до демо, навіть до початку роботи над AI: без FK-зв'язку вантажу з власником неможливо коректно побудувати ані trust scoring, ані персоналізований matching, ані журнал переговорів — вся AI-підсистема з розділу 7 неявно покладається на те, що це буде виправлено в Фазі 2.

